



武汉大学
WUHAN UNIVERSITY

F7LY-OS

内核设计与优化实现文档

队伍名称: F7LY

队伍成员: 曹子宸、郑喆宇、何佳怡

指导老师: 蔡朝晖

武汉大学计算机学院

2026 年 8 月

目录

第一章 赛题目标与证据范围	1
1.1 题目要求	1
1.2 项目与测试负载	1
1.3 文档版本与事实来源	1
1.4 证据分级	2
第二章 项目文档与 Agent 协作流程	3
2.1 文档分层原则	3
2.2 agent_docs 的使用	3
2.3 plan_docs 的生命周期	4
2.4 Agent 的标准开发闭环	4
第三章 脚本、工具与性能诊断	6
3.1 scripts 与 tools 的边界	6
3.2 scripts 目录使用地图	6
3.3 tools 目录使用地图	7
3.4 PERF_DIAG 与 f7ly-perf	7
3.5 BuildStorm 探针模式	8
3.6 日志与实验身份	9
第四章 问题定位与根因分析	10
4.1 固定测量边界	10
4.2 调度器空闲扫描与初始选核	10
4.2.1 现象	10
4.2.2 观测与根因	10
4.3 路径重复解析与 ext4 热点	10
4.3.1 现象	10
4.3.2 观测与根因	11
4.4 futex 与 PCB 锁序 ABBA	11
4.4.1 现象	11
4.4.2 观测与根因	11
4.5 地址空间、ASID 与远端 TLB	11

4.5.1 现象	11
4.5.2 观测与根因	11
4.6 大 VMA 与大文件偏移截断	12
4.6.1 现象	12
4.6.2 观测与根因	12
4.7 无效优化的识别与回退	12
第五章 修复与优化设计实现	13
5.1 调度：从全表扫描到活跃候选	13
5.2 futex：统一锁序与精确候选	13
5.3 地址空间：共享 mm 拥有 ASID 与最终清理权	13
5.4 文件系统：权威失效、连续 extent 与缩短排他区	14
5.5 大范围映射：统一 64 位契约	14
5.6 可选性能观测框架	14
5.7 三层验证门禁	15
第六章 实验分析与性能结果	16
6.1 实验口径	16
6.2 RV 并行编译启动阶段	16
6.3 双架构完整冷构建	16
6.4 Buddy 空闲页统计窄测	17
6.5 SMP 与 TLB 辅助结果	17
6.6 正确性回归	17
6.7 实验限制与复现改进	18
第七章 AI 工具使用说明与开发复现	19
7.1 AI 工具与责任边界	19
7.2 AI Agent 的输入与产出	19
7.3 可复用提示词模板	20
7.4 一次优化过程的复现步骤	20
7.4.1 1. 固定仓库和环境	20
7.4.2 2. 验证诊断框架	20
7.4.3 3. 建立有界基线	21
7.4.4 4. 采样并形成根因证据	21
7.4.5 5. 实现并运行窄验证	21

7.4.6 6. 运行普通内核 A/B	22
7.4.7 7. 更新计划并等待人工验收	22
7.5 AI 贡献披露样例	22
第八章 附录	23
8.1 附录 A: 最小复现命令	23
8.1.1 构建与配置	23
8.1.2 性能工具	23
8.1.3 定向回归	23
8.1.4 BuildStorm 正式探针	23
8.2 附录 B: 提交与设计索引	24
8.3 附录 C: 评审证据清单	24
8.4 附录 D: 文档维护规则	25
8.5 附录 E: 当前结论	25

第一章 赛题目标与证据范围

1.1 题目要求

本文件面向“2.3 内核设计优化实现文档”人工评审项，目标不是重复罗列 F7LY-OS 已实现的系统调用，而是说明团队怎样发现 BuildStorm 等真实负载中的内核问题、怎样设计并实现通用修复、怎样用固定边界比较优化前后性能，以及怎样使用 AI Agent、项目文档和自动化工具复现开发过程。

表 1-1 评分项与本文证据对应关系

分值	评审点	本文证据
6 分	问题定位与根因分析	第 4 章给出调度扫描、路径解析、futex 锁序、TLB/地址空间和大 VMA 等案例，并区分现象、观测和根因。
6 分	修复或优化设计实现	第 5 章按对象所有权、锁序、缓存一致性和平台边界说明设计与关键实现。
4 分	修改前后实验分析	第 6 章记录计时边界、历史 A/B 数据、加速比、验证范围和当前限制。
4 分	AI 使用与可复现步骤	第 2、3、7 章说明 Agent 文档路由、plan_docs 生命周期、脚本/工具使用、提示词模板和人工验收。

1.2 项目与测试负载

F7LY-OS 是面向 Linux ABI 的 C++23 freestanding 内核，支持 RISC-V 与 LoongArch。决赛阶段的 BuildStorm 在 glibc rootfs 中运行 Rust/Cargo 冷构建，持续制造多进程、多线程、futex、pipe、动态 ELF、mmap/COW、跨核 TLB、ext4 元数据和大量中间文件压力。与单条 LTP 相比，它更容易暴露系统级瓶颈和生命周期错误，因此本文以 BuildStorm 为主要优化案例，同时用 SMP、TLB、futex、ext4 和大内存专项建立窄验证闭环。

CAgent、LTP 和四组合 scoreboard 用于检查 Linux 语义和回归范围；BuildStorm 完整冷构建用于验证综合压力和最终编译时间。窄测只能证明对应不变量，不能替代完整评测；一次完整构建也不能替代多轮外部评测。

1.3 文档版本与事实来源

本文在仓库提交 32bebd65 的文档结构上整理，排版复用 docs/report-src/2026-final-final/。技术事实按以下优先级核对：当前源码和构

建配置、Git 提交、agent_docs/、已完成的 plan_docs/、结构化测试结果与 QEMU 日志。历史计划中的未完成项不会被写成当前 PASS，旧日志中的性能数字也不会被描述成本次重新运行所得。

本文引用的主要优化提交包括：

- ad43daf7：调度、路径和 ext4 等多维热路径优化，RV 并行 rustc 启动显著提前；
- 22f58137：完善 SMP、futex、地址空间与 BuildStorm 并发调试支持；
- 7d5a303c：收口 BuildStorm 依赖的进程和 VFS 容量路径；
- d092dec1：加入性能诊断、文件页缓存、批量回收等优化；
- 28ea8f25：完成 /proc/f7ly/perf、采样和用户态 f7ly-perf；
- dbec6fa5：继续优化内存、调度与 ext4，并整理完整冷构建结果；
- 1966b4ac：删除经验证无效或不合适的性能操作；
- f4aa440b：修复 LoongArch 大 VMA 和大文件偏移链路中的 32 位截断。

1.4 证据分级

本文统一使用三类证据，防止把“能编译”误写成“测例通过”：

表 1-2 F7LY-OS 证据分级

级别	典型入口	可证明内容
静态门禁	make build、 git diff -- check 无浮点检查	代码可编译、画像边界和 ABI 门禁成立；不能证明运行语义。
定向回归	futex、TLB、 ext4、SMP、VMA 专项	某一并发不变量或错误路径已覆盖；不能替代完整冷构建。
综合验收	官方 BuildStorm、 CAgent、Docker harness	真实工作负载可以完成；性能结论还需同条件重复数据。

本次工作只新增并编译本文，没有重新执行长时间 BuildStorm 或 Docker workflow。第 6 章的数据来自仓库既有实验记录，并保留其原始测试范围和限制。

第二章 项目文档与 Agent 协作流程

2.1 文档分层原则

项目没有把所有规则塞进单个提示词，而是用 `AGENTS.md`、`agent_docs/`、`plan_docs/`、Git 和日志分别承担规则、知识、计划、实现历史和运行证据。AI Agent 开始任务时先建立正确上下文，再运行工具和修改代码；这样可以减少基于过期事实直接生成补丁的风险。

表 2-1 项目协作信息分层

载体	职责	使用规则
<code>AGENTS.md</code>	全局入口与安全边界	每次任务先读；确认语言、Git 权限、Python 环境、日志位置、危险操作和文档路由。
<code>agent_docs/</code>	长期有效的项目知识	按任务选择架构、开发调试或 <code>scoreboard</code> 文档；源码与 Git 变化后同步维护。
<code>plan_docs/</code>	具体任务的目标与过程	记录范围、基线、假设、阶段、验收和未完成项；不能替代真实日志。
<code>scripts/</code>	可重复执行的编排	负责构建、临时镜像、QEMU、日志、超时和结果判定；优先使用已有脚本。
<code>tools/</code>	测试程序与分析器	提供 <code>guest</code> 专项程序、性能 CLI 和日志解析器，由脚本编译、注入或调用。
Git 与日志	变更与运行证据	Git 解释代码为什么变化；日志证明具体命令在具体版本上的结果。

2.2 `agent_docs` 的使用

`agent_docs/project_architecture.md` 用于定位模块和保护设计边界。涉及 `mmap` 时要同时检查 `syscall`、`VMA`、缺页、`COW` 和退出释放；涉及 `pthread/futex` 时要同时检查 `clone flags`、等待队列、信号和架构 `TLB`。Agent 应先从架构地图确定权威对象和不变量，再进入具体函数，避免只在测例报错位置添加特判。

`agent_docs/development_debugging.md` 提供构建画像、QEMU、GDB、镜像准备、日志命名和验证门槛。内核修改后至少构建对应画像；公共平台边界变化要构建四画像。长输出写入 `logs/run/`，聊天和报告只摘录结构化终态与错误上下文。

`agent_docs/scoreboard.md` 与 `scoreboard/README.md` 用于维护 RISC-V/LoongArch × musl/glibc 四组合状态。只有日志出现明确 `PASS`、`LTP summary` 为零

失败或用户提供有效证据时才能写 `PASS`；构建成功、没有 `panic` 或其他架构通过都不能推导当前组合通过。

`agent_docs/platform_refactor_design.md` 与 `board_bringup_beginner.md` 属于按需资料。前者解释平台画像、`backend` 和设备边界，后者用于 VisionFive2/2K1000 实板首次启动。它们不应在普通 `syscall` 调试中无条件加载，但相关任务必须阅读。

2.3 `plan_docs` 的生命周期

`plan_docs` 是工程任务账本，不是自动生成的总结目录。一个有效计划至少写清楚：目标和非目标、当前基线、已知风险、分阶段实现、每阶段验收命令、结果状态和外部验收边界。

Agent 使用计划的推荐顺序如下：

1. 读取现有同主题计划，确认任务是否已经完成、被替代或仍有开放项；
2. 用 `git status --short` 和近期提交校准计划中的历史描述；
3. 开始新方向时新增计划，继续同一方向时原位更新，避免建立多份互相冲突的任务事实；
4. 每个假设记录对应证据和反证条件，例如“远端 TLB 等待持续增长”而不是“感觉 TLB 很慢”；
5. 完成后在对应位置写“已完成，待验收”，保留未完成项和测试范围；
6. 人工验收后才能把结果用于正式报告或 `scoreboard`。

表 2-2 `plan_docs` 任务生命周期

阶段	计划内容	退出条件
基线	版本、环境、工作量、失败位置	同一命令可以稳定复现。
定位	候选瓶颈、采样手段、对照实验	至少一项证据能区分候选根因。
实现	设计不变量、修改模块、回退方案	代码通过静态门禁和窄验证。
验收	A/B、回归矩阵、日志路径	结构化终态完整，未完成项显式保留。
收尾	完成待验收、文档同步、人工决定	不自动 <code>commit</code> 、不夸大证据范围。

2.4 Agent 的标准开发闭环

一次性能优化任务采用以下闭环：

读取 AGENTS.md

- > 按任务读取 agent_docs
- > 检查 Git 与 plan_docs 基线
- > 用 scripts 固定工作量并保存日志
- > 用 tools/采样/GDB 形成根因证据
- > 修改权威对象与不变量
- > 静态门禁 + 定向回归
- > 同条件 A/B + 综合验收
- > 更新计划/scoreboard/正式文档
- > 人工 review 决定是否合入

其中任何一步失败都应停在当前层次修复，不能通过跳过 crate、复用旧 target、修改 uptime、降低工作量或伪造 PASS 标记跨过失败点。

第三章 脚本、工具与性能诊断

3.1 scripts 与 tools 的边界

`scripts/` 负责“怎样安全、重复地运行”：检查依赖、选择画像、构建内核、复制临时镜像、注入程序、启动 QEMU、设置超时、保存日志并判断结构化终态。`tools/` 负责“运行什么或怎样分析”：包含可交叉编译的 C/C++ 专项程序、`f71y-perf`、`LTP parser/ranker` 和辅助修补工具。

这个边界使 `guest` 测试逻辑可审计，同时把容易误写原始镜像、混淆架构或遗失日志的宿主操作集中在脚本中。`Agent` 应优先调用脚本，而不是临时拼接一条不可追踪的 QEMU 命令。

3.2 scripts 目录使用地图

表 3-1 scripts 使用地图

目录/脚本	用途	使用注意
<code>scripts/images/</code>	准备或恢复 QEMU 镜像	<code>prepare-qemu-image.sh</code> 是普通运行入口； <code>restore-sdcards.sh</code> 会覆盖工作镜像，必须人工确认。
<code>scripts/mount/</code>	挂载 RV/LA 初赛或决赛 rootfs	用于检查磁盘内容；挂载点和 loader 约定见开发调试文档。
<code>scripts/run/</code>	QEMU 回归、专项和性能探针	默认使用临时副本或 snapshot；完整输出进入 <code>logs/run/</code> 。
<code>scripts/selfbuild/</code>	准备 selfhost rootfs 和分级自编译	需要较长时间、磁盘与外部依赖；不能替代官方 BuildStorm。
<code>scripts/board/</code>	2K1000 实板下板/启动辅助	只在明确的实板任务中使用。
<code>scripts/dev/</code>	代码统计和 Git 烟测	工程辅助，不作为内核功能证据。
<code>generate_perf_symbols.sh</code>	为诊断内核生成两阶段稳定符号表	由构建系统自动调用，不应手工改写生成物。
<code>check_kernel_no_fp.sh</code>	检查 soft-float ABI、内核浮点/向量指令与 helper	性能诊断代码也必须通过这一门禁。

常用窄验证入口包括：

```
scripts/run/futex_short_test.sh --arch all --qemu-cpus 8
scripts/run/tlb_shootdown_test.sh --arch all --rounds 200
scripts/run/ext4_concurrency_test.sh --arch all --qemu-cpus 8 --qemu-
```

```
mem 8G
scripts/run/smp_stress_ng.sh --qemu-cpus 8 --worker-list 1,2,4,8 --
runs 3
```

`smp_cpu_bench.sh` 的吞吐缩放结论要求对应架构的原生 KVM 宿主；x86_64 上的跨架构 TCG 只能观察 guest 多核行为，不能冒充原生硬件扩展性能。

3.3 tools 目录使用地图

表 3-2 tools 使用地图

路径	内容	与脚本的关系
tools/perf/	f71y_perf.cc 与 native test	编译为双架构静态 CLI，读取 /proc/f71y/perf； native test 验证解析与聚合算法。
tools/smp/	CPU 吞吐、TLB shutdown、mm 释 放竞态	由对应 scripts/run/ 脚本交叉编译并注入临时 rootfs。
tools/proc/	futex/线程退出短测	由 futex_short_test.sh 编译和运行。
tools/fs/	ext4 并发一致性测 试	由 ext4_concurrency_test.sh 运行并在关机后 执行只读 fsck。
tools/ltp/ judge/	LTP 日志解析、排 序、组合流水线	用于批量结果分析，不直接修改内核 PASS。
tools/ltp/ scoreboard/	历史 LTP 清单转换 工具	当前协作状态以顶层 scoreboard/ 为准。
patch_loongarch_ libctest_llsc.sh	LoongArch libctest LL/SC 辅助补丁	只在对应用户态测试环境明确需要时使用。

3.4 PERF_DIAG 与 f71y-perf

普通内核默认 `PERF_DIAG=0`，不承担采样代码和 frame pointer 开销。显式使用 `PERF_DIAG=1` 时，构建目录和内核产物增加 `-perf` 后缀，并启用统一指标表、per-CPU epoch 计数、syscall 耗时、timer/PMU 采样和可选调用链。

首先验证宿主侧解析算法和双架构 CLI：

```
make perf-native-tests
make perf-tools
```

然后运行不改变官方工作量的诊断 smoke：

```
PERF_DIAG=1 PROBE_MODE=perf-smoke \
bash scripts/run/buildstorm_perf_probe.sh riscv
```

```
PERF_DIAG=1 PROBE_MODE=perf-smoke \
bash scripts/run/buildstorm_perf_probe.sh loongarch
```

guest 中的主要命令为：

```
f7ly-perf status --json
f7ly-perf stat --interval-ms 1000 --count 3 --per-cpu --json
f7ly-perf top --backend auto --event cycles --frequency 100 \
--callgraph --duration 10 --limit 20 --json
f7ly-perf reset all
```

auto 后端在 PMU 不可用时回退到 timer。显式 pmu 失败是能力报告，不应伪造为采样成功；timer 热点适合判断函数级分布，但 QEMU TCG 下的采样值不能外推到原生硬件绝对性能。

3.5 BuildStorm 探针模式

scripts/run/buildstorm_perf_probe.sh 使用临时评测镜像，拒绝在已有 QEMU 竞争时启动，并将串口与宿主 vCPU 采样分别保存。PROBE_MODE 决定 guest 工作负载：

表 3-3 BuildStorm 性能探针模式

模式	用途	适用阶段
progress	固定窗口观察 crate 和产物进展	快速比较是否越过已知停顿点。
formal	空目标目录的正式冷构建和周期快照	最终 A/B；计时轮应使用普通内核。
teardown	256 MiB 地址空间退出回收延迟	定位 mm teardown。
filecache	同一大文件两轮并发遍历	验证文件页复用和底层读取降幅，要求 PERF_DIAG=1。
qemu-ram	大范围预留、激活和触页	定位大 VMA、回收和内层 QEMU ENOMEM。
free-count	高频空闲页统计	验证 Buddy 空闲计数复杂度。
perf-smoke	/proc/f7ly/perf ABI 与采样测试	诊断框架自身验收。
xtask	原始固定时长 BuildStorm 入口	历史进度探针。

正式诊断示例：

```
PERF_DIAG=1 PROBE_MODE=formal SKIP_CAGENT=1 \  
HOST_TIMEOUT=60m QEMU_MEM=8G QEMU_SMP=8 CHECK_E2FSCK=1 \  
bash scripts/run/buildstorm_perf_probe.sh riscv
```

最终计时必须再以 `PERF_DIAG=0`、同一镜像、同一 CPU/内存、空 target 和相同宿主负载重复运行。诊断数据用于找到瓶颈，普通内核时间才用于报告最终收益。

3.6 日志与实验身份

每轮 A/B 至少保存：Git commit 与 dirty 状态、完整命令、架构和 vCPU、QEMU/工具链版本、rootfs SHA-256、开始时间、退出码、结构化终态和原始日志路径。推荐在运行前记录：

```
git rev-parse HEAD  
git status --short  
qemu-system-riscv64 --version  
riscv64-linux-gnu-g++ --version  
sha256sum images/oscomp-final-riscv64.img
```

正式结果以唯一的 `BUILDSTORM_FORMAL_RESULT ok=true` 和 `BUILDSTORM_COMPILE ... elapsed_s=... cores=... bytes=...` 为准；仅有 QEMU 正常关机、出现若干产物或没有 panic 都不足以判定成功。

第四章 问题定位与根因分析

4.1 固定测量边界

BuildStorm 的性能定位先固定官方工作量：使用决赛 glibc rootfs、真实 guest 单调时钟、指定 vCPU/内存，删除正式目标架构的旧 target，不跳过 crate、不复用缓存产物、不修改 `/proc/uptime`。宿主 CPU 采样、GDB 全核栈和 `PERF_DIAG` 只作为旁路观测，不改变 Cargo 命令和结果判定。

定位过程按“现象—观测—排除—根因”推进。某条路径占用时间较多不等于它就是错误根因；例如 `ext4 ticket` 持续推进说明存在锁竞争，但不能据此宣称死锁。只有对照实验、重复栈或结构化计数能够区分候选原因时才修改生产代码。

4.2 调度器空闲扫描与初始选核

4.2.1 现象

早期 RV 8G/8 vCPU 冷构建在 900 guest 秒窗口内没有进入有效 `Compiling`，target 中只有少量 `deps` 文件；宿主却观察到 8 个 QEMU vCPU 线程长期占用 CPU。

4.2.2 观测与根因

宿主线程采样和 guest 进程快照表明，vCPU 时间大量消耗在空闲调度扫描，而不是 `rustc` 并行执行。旧调度器反复扫描全局 PCB 表并获取无关任务锁；新任务按轮转相位分配 home CPU，短命 `shell/build script` 还会让重型 `rustc` 与 `cargo` 撞在同一核。

根因不是“CPU 数不够”，而是候选集合和执行权发布缺少低成本索引：空闲 CPU 为寻找少数 `RUNNABLE` 任务付出全表扫描成本，新任务初始放置也没有使用各 CPU 的实际活跃压力。

4.3 路径重复解析与 ext4 热点

4.3.1 现象

调度优化后能够启动 `rustc`，但大量 `crate` 仍长时间停留在源码、动态库和元数据查找阶段，`deps` 产物增长缓慢。

4.3.2 观测与根因

`Cargo/rustc` 会反复访问 `/work`、`/tmp`、工具链目录和相同共享库。旧 `root` 打开路径对每级前缀分别执行 `resolve` 与 `stat`，存在性查询和类型查询再次走完整路径；普通读还可能频繁写 `atime`。`ext4` 全局排他区覆盖过多直接数据 I/O，`bcache` 与路径元数据热点缺少权威失效规则。

根因是路径组件和文件数据连续性没有贯穿 `VFS`、`ext4` 和块层：重复的语义查询造成 I/O 与锁放大，而简单增加缓存若没有 `create/rename/link/unlink` 的权威失效，只会引入过期目录项。

4.4 futex 与 PCB 锁序 ABBA

4.4.1 现象

`BuildStorm` 一度稳定停在 `Compiling core`。`CPU` 仍在线，进程存在，但 `Cargo/rustc` 不再产生新进度。

4.4.2 观测与根因

修复前后的 8 vCPU `GDB` 全核栈形成了闭环证据：一个 `CPU` 持 `futex` 全局锁等待 `PCB` 锁，另一个 `CPU` 沿 `wake`/退出路径持 `PCB` 锁再等待 `futex` 锁。`WAIT` 路径还存在检查用户值与发布等待 `key` 之间的窗口，`WAKE` 可能扫描大量无关 `PCB`。

根因是 `WAIT`、`WAKE`、超时和退出清理没有遵循唯一锁序，也没有把等待 `key` 作为原子可预筛选状态发布。这是通用并发错误，不是 `core crate` 特例。

4.5 地址空间、ASID 与远端 TLB

4.5.1 现象

在 8 vCPU `BuildStorm` 中，间隔计数显示远端 `TLB shutdown` 累计等待持续增长；多线程共享 `mm` 的地址空间修改和退出回收成本显著。

4.5.2 观测与根因

旧实现把 ASID 与线程 PCB 绑定，CLONE_VM 线程虽然共享页表，却可能拥有不一致的 TLB 身份；页表变化还会广播到没有运行该 mm 的 CPU。teardown 逐页撤销并同步，使退出路径重复承担锁、IPI 和页表清理成本。

根因是 TLB 身份和同步范围没有跟随共享地址空间对象：ASID、generation、活跃 CPU mask 和最终销毁权应属于 ProcessMemoryManager，而不是单个线程。物理页只有在目标 CPU 确认失效后才能回收。

4.6 大 VMA 与大文件偏移截断

4.6.1 现象

LoongArch 的 Rust 冷编译完成后，内层 QEMU 预留 8 GiB guest RAM 报 ENOMEM；RV 的大偏移文件 mmap/fork 链路也存在读错页风险。

4.6.2 观测与根因

用 PROT_NONE 预留、2 MiB 对齐、MAP_FIXED 激活、裁边和首尾触页构造等价窄测后，旧内核在大映射上稳定失败。审计发现单个 VMA 长度曾经过 32 位 int，公共后备缺页又把“文件基准偏移 + VMA 页偏移”收窄。

根因是地址范围和文件位置在 syscall、VMA 拆分/合并、缺页与文件后端之间没有保持统一 64 位契约。仅修改 syscall 参数类型不能修复后续链路中的再次截断。

4.7 无效优化的识别与回退

开发过程中尝试过运行期激进任务迁移、若干 ext4 快速分配/缓存操作和过度特化路径。部分方案破坏 guest sleep/计时稳定性、缓存一致性或没有形成可重复收益，因此在 1966b4ac 等提交中删除。

性能优化的退出条件不仅是“更快”，还包括：双架构语义不退化、锁序和对象所有权可解释、定向回归通过、正式计时可重复。不能满足这些条件的候选优化应回退，并在计划中记录失败原因。

第五章 修复与优化设计实现

5.1 调度：从全表扫描到活跃候选

调度器为每个 home CPU 维护可运行任务位图和原子压力计数。任务进入或离开 RUNNABLE 时更新索引；调度器先读取候选位图，再获取对应 PCB 锁确认状态和唯一运行权。初始选核只比较各 online CPU 的活跃压力，复杂度从反复扫描全 PCB 表收敛为 $O(\text{NUMCPU})$ 原子读取。

跨核唤醒通过语义化 `kick_cpu` / IPI 通知目标 CPU，不依赖周期性扫描碰巧发现任务。运行期不启用激进迁移：home CPU 和执行权规则保持稳定，避免为了某一轮吞吐破坏 affinity、sleep 或 guest 时间行为。

5.2 futex：统一锁序与精确候选

futex key 根据 private/shared 属性转换为 mm 私有键或物理页键，并登记到固定 bucket。WAIT 和 WAKE 统一遵循“futex bucket/全局锁 → PCB 锁”的顺序：WAIT 在锁内重新读取用户值，值不匹配立即返回 `-EAGAIN`；匹配后原子发布 key 和等待状态。WAKE 只扫描相同 bucket 和 key 的候选，并只唤醒能取得执行权的任务。

robust list、超时、信号中断、`FUTEX_REQUEUE` 和 `CLONE_CHILD_CLEARTID` 复用同一状态模型，防止退出清理重新引入反向锁序。该设计同时解决 ABBA、丢唤醒和惊群扫描，而不是为 BuildStorm 返回固定成功。

5.3 地址空间：共享 mm 拥有 ASID 与最终清理权

`ProcessMemoryManager` 是页表、VMA、VmObject、ASID、TLB generation、活跃 CPU mask 和引用计数的权威所有者。`CLONE_VM` 线程共享同一 mm/ASID；fork 创建独立 mm；exec 成功后原子替换 mm。PCB 在归还前先摘除 mm 指针，本次原子引用递减结果唯一决定谁执行最终 `free_all_memory()`。

页表变化只向当前运行该 mm 的 CPU 发送定向 range/ASID shutdown。本地和远端完成失效、generation 得到确认后，才允许释放旧 PTE、COW 页或整个地址空间。teardown 先批量撤销 VMA/PTE，再集中释放页表与后端引用，减少逐页同步。

RISC-V 后端使用 SBI remote fence；LoongArch 使用 IOCSR IPI 和 request/ack generation，并按双页 TLB 粒度覆盖末端区间。架构差异留在 HAL，通用 VMA 代码只消费统一的同步契约。

5.4 文件系统：权威失效、连续 extent 与缩短排他区

路径组件缓存同时保存正向和负向结果，但 create、rename、link、unlink 和目录 rename 必须从权威目录项操作位置使相关缓存失效。root 打开路径合并前缀 resolve/stat，symlink 使用单遍扫描；普通读采用 relatime 条件更新，减少无意义的 atime 写放大。

ext4 挂载锁采用可重入 FIFO 睡眠锁，同 owner 嵌套只增加深度，其他线程按顺序睡眠。bcache 使用独立锁和 O(1) dirty 链，完整块数据 I/O 移出全局元数据排他区；连续 extent 查询、分配、I/O 和 cache 一致性失效按 run 批量执行。这样缩短锁持有时间，同时保留 fsync、close、rename 和读后可见性。

文件页缓存按文件对象和页偏移复用 clean 页面。分配失败时可以有界淘汰 clean 文件页再重试；dirty 页不能被当作普通可回收页丢弃。exec、mmap 和重复动态库读取因此可以共享底层页，但 truncate 和文件修改仍必须使对应页失效并保持 SIGBUS/EOF 语义。

5.5 大范围映射：统一 64 位契约

VMA 长度、起止地址、页偏移和文件偏移统一使用可覆盖用户地址空间的 64 位类型。拆分、合并、扩展、mremap、fork 克隆、后备缺页和“文件基准偏移 + VMA 页偏移”都显式检查加法溢出和底层接口边界。

上层 mmap/fork 无条件调用统一的用户范围准备入口；只有 LoongArch 在入口内部按 TLB refill 契约预建必要页表骨架。架构行为不再通过上层重复分支实现，避免 RV 和 LA 的普通映射路径逐渐分叉。

5.6 可选性能观测框架

PERF_DIAG=1 生成独立诊断内核，普通内核完全不暴露相关 proc ABI。诊断框架采用静态指标注册和 per-CPU 计数，避免热点路径争用一个全局统计锁；syscall 记录 count/time ticks，profile 支持 timer/PMU、符号化热点和可选调用链。

符号表通过首次链接 ELF 生成，二次链接后校验 text 地址稳定。f71y-perf 以 /proc/f71y/perf v1 TSV 为数据源，将内核 ABI 与人类可读/JSON 输出分开。frame pointer、采样和统计开销只存在于 -perf 产物，最终正式计时必须回到普通内核。

5.7 三层验证门禁

每次优化按以下顺序验收：

1. 双架构或四画像构建、git diff --check、内核无浮点/向量门禁；
2. 与改动对应的 futex、TLB、VMA、ext4、SMP 或性能 ABI 窄测；
3. 空 target 的完整 BuildStorm、CAgent 连续回归及需要时的 Docker harness。

某层失败时不继续扩大工作量。窄测通过但完整构建失败时，结论应写成“对应不变量已验证，综合负载仍开放”，而不是把计划项直接标为全部完成。

第六章 实验分析与性能结果

6.1 实验口径

正式 BuildStorm 使用 `guest /proc/uptime` 对 `cargo xtask arceos build` 计时，预构建 `tg-xtask` 不计入正式时间；正式目标架构的旧 `target` 在计时前删除。结果必须同时包含 `ok=true`、`guest elapsed`、`vCPU` 数和非空产物字节数。

诊断轮允许 `PERF_DIAG=1`、GDB 或宿主 CPU 采样；最终性能轮使用普通内核。A/B 必须保持 `rootfs`、`Cargo` 命令、`vCPU`、内存、`QEMU` 加速方式和宿主竞争状态一致。历史结果中没有完整保存这些字段的部分，在本章明确标注为限制。

6.2 RV 并行编译启动阶段

2026-07-29 的同一 RV 8G/8 vCPU 冷诊断中，旧路径约 415 guest 秒才启动并行 `rustc`，优化后约 107 秒进入并行 `rustc`：

加速比： $415 / 107 \approx 3.88$ 。

耗时降幅： $(415 - 107) / 415 \times 100\% \approx 74.2\%$ 。

这一数据反映“进入并行 `rustc` 的启动阶段”，不能当作完整 BuildStorm 编译加速比。优化后 8 个 `rustc` 曾实际分布到 CPU0—7，说明收益来自调度候选、初始选核和路径开销共同改善，而不是只提前打印日志。

6.3 双架构完整冷构建

2026-08-14 的历史正式探针记录如下：

表 6-1 BuildStorm 完整冷构建历史 A/B 结果

架构	vCPU	优化前/s	优化后/s	加速比	耗时降低
RV64	8	3223.75	1693.23	1.904×	47.5%
LA64	12	2564.93	1367.35	1.876×	46.7%

RV 产物为 1,681,000 字节，LA 产物为 1,714,568 字节；两轮均记录 `ok=true`，没有跳过 `crate`、复用旧 `target` 或修改 `guest` 计时。对应历史日志为：

- `logs/run/output_r_20260814-191332_buildstorm-perf.txt`；
- `logs/run/output_l_20260814-194725_buildstorm-perf.txt`。

这些日志位于本地 `logs/run/` 且默认不提交 Git。正式交付时应同时提供结构化摘要、日志 SHA-256 或单独证据包，避免评审者只能看到正文数字。

6.4 Buddy 空闲页统计窄测

大内存回收路径曾在 `sysinfo` 中扫描 Buddy 状态计算空闲页。改为在分配/释放时 $O(1)$ 记账后，LoongArch 在 32 MiB 碎片化、5000 次 `sysinfo` 条件下，单次耗时由 215,120 ns 降至 12,553 ns：

加速比： $215120 / 12553 \approx 17.1$ 。

该窄测只证明空闲页统计复杂度下降，不应直接乘到 BuildStorm 总时间上。它与完整构建共同说明优化既处理可观测系统信息的热点，也没有破坏综合负载。

6.5 SMP 与 TLB 辅助结果

既有复现记录中，RV 静态 CPU 基准的 1/2/4/8 worker 吞吐为 11524.665、24957.165、50653.834、99504.986 events/s，8 worker 相对 1 worker 为 8.634×；LA 对应为 16101.520、33044.438、66386.127、123023.106 events/s，8 worker 为 7.640×、效率 95.51%。

RV 真实 stress-ng 的 1/2/4/8 worker 吞吐为 35.270、69.190、142.120、271.240 bogo ops/s，8 worker 为 7.690×、效率 96.13%。这些结果用于证明 guest 能利用多个 CPU，不直接代表跨架构 TCG 的原生硬件绝对性能。

双架构 8 GiB TLB 专项覆盖 `mprotect` 降权、预期 fault、`munmap` 和 `MAP_FIXED` 重映射。历史 20 轮结果中，RV managed pages 为 1,751,684，LA 为 1,947,687，均完成 40 次预期 fault、20 次 `mprotect` 和 20 次 `munmap`。

6.6 正确性回归

性能结果之外，相关优化还使用以下门禁防止负收益：

- `futex`/线程退出短测覆盖 `WAIT`、`WAKE`、超时和 `clear-tid`；
- `ext4` 并发短测覆盖 `write/fsync/rename/read/unlink`，关机后对临时镜像执行只读 `e2fsck -fn`；
- 大 VMA 测试覆盖 8 GiB 预留、首尾触页、中段 `mprotect` 和完整 `munmap`；
- 大文件测试覆盖 3 GiB 文件偏移 `mmap + fork`；
- 双架构 CAgent、定向 LTP 和内核构建用于检查 Linux ABI 回归。

6.7 实验限制与复现改进

当前历史数据仍有以下限制：完整冷构建主要保留单轮结果；LA 使用 12 vCPU，不能与 8 vCPU 结果混成一张无条件横向对比；部分旧基线只在计划中记录，没有随 Git 提交原始日志和镜像哈希。因此本文给出的加速比可说明已观察到的收益，但最终评审复现应补充至少三轮同条件结果，并报告中位数、最小/最大值和异常轮原因。

推荐的结果表字段为：`arch`、`commit`、`dirty`、`profile`、`perf_diag`、`qemu_version`、`accel`、`vcpus`、`memory`、`image_sha256`、`elapsed_s`、`artifact_bytes`、`result`、`log_sha256`。这能把性能数字和可重放实验身份绑定起来。

第七章 AI 工具使用说明与开发复现

7.1 AI 工具与责任边界

项目在 2026 年开发过程中使用 VSCode/Codex 交互环境中的 Codex，相关大模型为 GPT-5。AI Agent 用于阅读代码与 Git 历史、定位模块、生成候选补丁、构造窄测、调用脚本、分析日志、整理 scoreboard 和维护文档草稿。

本项目把 OpenAI 官方文档中“明确上下文、硬约束、成功标准和授权边界，并只暴露当前任务所需工具”的通用建议落实为仓库内契约：**AGENTS.md** 定义授权边界，**agent_docs/** 提供按任务加载的上下文，**plan_docs/** 保存阶段和证据，**scripts/** 与 **tools/** 提供可重复执行的受控入口。产品层原则可参见 [OpenAI 官方模型与工具调用指南](#)；审核时仍以本仓库中的命令、diff 和日志为准。

系统设计目标、比赛规则解释、是否接受方案、最终代码合入、测例是否计入通过、commit/push 和答辩材料由参赛队员负责。未经用户明确允许，Agent 不提交 commit、不推送、不重置历史；没有运行证据时不写 PASS；外部 Docker 完整 workflow 由用户决定何时执行。

7.2 AI Agent 的输入与产出

表 7-1 AI Agent 输入输出链

阶段	Agent 输入	可追踪产出
建立上下文	AGENTS.md、任务相关 agent_docs、Git 状态	任务边界、当前架构事实和不变量。
形成计划	用户目标、现有 plan_docs、基线日志	阶段、假设、风险和验收命令。
定位问题	scripts 输出、f71y-perf、GDB、源码	现象—证据—根因链，含被排除候选。
候选实现	权威对象、锁序、ABI 和架构边界	可 review 的 diff、中文关键注释和回退点。
验证	静态门禁、窄测、A/B、完整负载	日志路径、结构化终态、结果范围和开放项。
人工验收	diff、日志、正式报告	接受/拒绝决定；必要时提交与 scoreboard 更新。

7.3 可复用提示词模板

审核者可以在仓库根目录向 Codex 提交如下任务，以复现同类开发流程：

请按 AGENTS.md 工作。阅读与 BuildStorm 性能相关的 agent_docs、plan_docs/final_2026_support_gap_plan.md、近期 Git 提交和现有脚本。

目标：在不修改官方 Cargo 工作量、不复用旧 target、不修改 uptime 的条件下，定位 <架构> BuildStorm 在 <阶段> 的瓶颈。

先记录 Git/宿主/QEMU/rootfs 身份，使用已有 buildstorm_perf_probe 和 f7ly-perf 做有界诊断；给出“现象-观测-根因”证据后再修改代码。修改必须落在权威对象和通用不变量上，不写测例特判。

完成后依次运行：对应画像构建、无浮点门禁、最窄专项、同条件 A/B。长日志写 logs/run，只汇报结构化终态。不要 commit、push 或运行完整 Docker workflow；将结果和未完成项回填原计划，等待人工验收。

任务中的 <架构>、<阶段>、CPU/内存和最大运行时间必须明确。若没有足够信息，Agent 可以先做只读检查，但不能自行扩大到破坏性镜像操作或外部提交。

7.4 一次优化过程的复现步骤

以下步骤复现“BuildStorm 进度慢或停在 crate 编译”类任务；命令默认在仓库根目录执行。

7.4.1 1. 固定仓库和环境

```
git status --short
git log -5 --pretty=fuller --stat
git rev-parse HEAD
qemu-system-riscv64 --version
sha256sum images/oscomp-final-riscv64.img
```

如果工作区已有用户改动，Agent 先读 diff，不能覆盖。基线与候选内核最好在独立 worktree 构建，并保留完整 commit；不要依赖模糊的“旧内核”文件名。

7.4.2 2. 验证诊断框架


```
make perf-native-tests
PERF_DIAG=1 PROBE_MODE=perf-smoke \
bash scripts/run/buildstorm_perf_probe.sh riscv
```

期望看到唯一 `PERF_SMOKE_RESULT ok=true`。若 `PMU unavailable`，`auto` 回退 `timer` 属于正常能力协商。

7.4.3 3. 建立有界基线

```
PROBE_MODE=progress SKIP_CAGENT=1 HOST_TIMEOUT=10m \
QEMU_MEM=8G QEMU_SMP=8 \
bash scripts/run/buildstorm_perf_probe.sh riscv
```

记录最后 `crate`、产物数、`rustc/cargo` 状态、宿主 `vCPU` 分布和 `panic/TLB timeout`。若目标是完整时间，改用 `PROBE_MODE=formal` 并给足超时；`progress` 不能作为完整 `PASS`。

7.4.4 4. 采样并形成根因证据

```
PERF_DIAG=1 PROBE_MODE=formal SKIP_CAGENT=1 HOST_TIMEOUT=20m \
QEMU_MEM=8G QEMU_SMP=8 \
bash scripts/run/buildstorm_perf_probe.sh riscv
```

结合 `/proc/f7ly/perf` 快照、宿主 `CPU` 日志和必要的 `GDB` 全核栈，比较 `scheduler`、`futex`、`TLB`、`ext4` 和 `mm teardown`。只有锁闭环、持续增长计数或同条件对照能支持根因结论；单个热点函数不能独立证明问题。

7.4.5 5. 实现并运行窄验证

例如修改 `futex` 锁序后运行：

```
make build PROFILE=riscv-qemu
make build PROFILE=loongarch-qemu
scripts/run/futex_short_test.sh --arch all --qemu-cpus 8
```

修改 `TLB/VMA` 或 `ext4` 时分别改用 `tlb_shutdown_test.sh`、`qemu-ram`、`ext4_concurrency_test.sh`，同时保留双架构构建。

7.4.6 6. 运行普通内核 A/B

诊断完成后以 `PERF_DIAG=0` 运行正式冷构建。基线和候选必须使用相同镜像、vCPU、内存和 Cargo 命令，各运行至少三轮；报告中使用中位数，并保留每轮数据。正式成功必须出现唯一 `BUILDSTORM_FORMAL_RESULT ok=true`。

7.4.7 7. 更新计划并等待人工验收

Agent 将根因、实现、命令、日志、A/B 和限制回填原 `plan_docs`，标记“已完成，待验收”。只有人工确认 diff 和日志后，才决定是否提交以及是否更新正式报告或 scoreboard。

7.5 AI 贡献披露样例

每个进入正式文档的 AI 辅助优化可以附一条记录：

表 7-2 AI 辅助优化披露字段

字段	记录内容
日期与版本	会话日期、Codex/GPT-5、仓库 commit、dirty 状态。
原始任务	用户目标与禁止事项，保留提示词或等价摘要。
Agent 操作	读取文档、调用脚本/工具、形成的候选根因和补丁文件。
人工决策	接受、拒绝或回退了哪些方案，理由是什么。
验证证据	构建、窄测、完整负载、日志和结构化结果。
证据范围	哪些已验证，哪些仍需重复长测或官方 harness。

这种记录让审核者能够沿“提示词—工具调用—代码 diff—日志—人工决策”重放开发过程，同时避免把 AI 生成候选方案误写成无人审核的最终设计。

第八章 附录

8.1 附录 A：最小复现命令

8.1.1 构建与配置

```
make profiles
make print-config PROFILE=riscv-qemu MODE=evaluation
make build PROFILE=riscv-qemu
make build PROFILE=loongarch-qemu
make build PROFILE=riscv-visionfive2
make build PROFILE=loongarch-2k1000
```

8.1.2 性能工具

```
make perf-native-tests
make perf-tools
PERF_DIAG=1 PROBE_MODE=perf-smoke \
  bash scripts/run/buildstorm_perf_probe.sh riscv
```

8.1.3 定向回归

```
scripts/run/futex_short_test.sh --arch all --qemu-cpus 8
scripts/run/tlb_shootdown_test.sh --arch all --rounds 200
scripts/run/ext4_concurrency_test.sh --arch all --qemu-cpus 8 --qemu-
mem 8G
scripts/run/smp_stress_ng.sh --qemu-cpus 8 --worker-list 1,2,4,8 --
runs 3
```

8.1.4 BuildStorm 正式探针

```
PROBE_MODE=formal SKIP_CAGENT=1 HOST_TIMEOUT=60m \
QEMU_MEM=8G QEMU_SMP=8 CHECK_E2F5CK=1 \
bash scripts/run/buildstorm_perf_probe.sh riscv
```

完整 Docker 并发 workflow 不属于 Agent 默认执行范围。完成窄验证后由用户决定是否运行：

```
bash build/oscomp-eval-20260624-232828/scripts/run_docker_autotest.sh
```

8.2 附录 B：提交与设计索引

表 8-1 优化提交索引

提交	主题	本文位置
ad43daf7	调度、路径、ext4 热路径与并行 rustc 启动	第 4—6 章
22f58137	SMP/futex/mm 并发与专项	第 4、5 章
7d5a303c	BuildStorm 进程与 VFS 容量收口	第 1、4 章
d092dec1	诊断、文件页缓存、批量回收	第 3、5 章
28ea8f25	/proc/f71y/、perf CLI、符号与采样	第 3、5 章
dbec6fa5	内存、调度、ext4 性能收口	第 4—6 章
1966b4ac	删除无效性能操作	第 4 章
f4aa440b	大 VMA 与大文件偏移 64 位修复	第 4、5 章

8.3 附录 C：评审证据清单

提交本文档时建议同时准备：

- 本文 PDF 与对应 Typst 源码；
- 当前内核 commit、dirty 状态和构建配置；
- A/B 每轮结构化 TSV/JSON，至少三轮；
- 原始日志或独立证据包及 SHA-256；
- rootfs 来源与 SHA-256、QEMU 和工具链版本；
- 关键提交 diff 或提交索引；
- AI 使用记录：提示词、模型、工具调用、人工接受/拒绝决定；

- 未完成项和外部验收边界。

8.4 附录 D：文档维护规则

本文源文件位于 `plan_docs/kernel_design_optimization/`，排版组件从 `docs/report-src/2026-final-final/` 导入。重新生成 PDF：

```
typst compile --root . \  
  plan_docs/kernel_design_optimization/main.typ \  
  plan_docs/kernel_design_optimization/f7ly-kernel-design-  
  optimization.pdf
```

当性能数字、脚本参数、工具 ABI 或最终报告结构变化时，应同步更新本文。若某项结果来源于历史日志而没有重新运行，必须继续标注“历史结果”，不能因为文档重新编译就改写为当前 HEAD 的新验收。

8.5 附录 E：当前结论

F7LY-OS 已经形成“文档路由—任务计划—脚本编排—专项工具—结构化日志—人工验收”的 AI 辅助内核开发流程。BuildStorm 优化展示了从系统现象到调度、锁序、地址空间、文件系统和大映射根因的逐层收口，也取得了明确的历史加速数据。

当前仍需补强的是性能结果的多轮统计和随提交交付的证据包。本文将这些限制保留为评审可见事实，并提供了补测与复现步骤，便于审核者在相同边界下重放开发过程。

已完成，待验收。